Tech365
Leveraging technology for growth

LEARN | BUILD | GROW

AWS

Databases on AWS

Choose, configure and use the right AWS database services for modern data engineering solutions.

Tech365
Leveraging technology for growth

LEARN | BUILD | GROW

Relational vs NoSQL Databases

Two different approaches. Both solve real problems.

	Relational Databases Structured data, fixed schema, SQL	NoSQL Databases Flexible data models, schema-on-write or schema-less
Data model	Table-based (rows and columns)	Various models (document, key-value, column-family, graph)
Schema	Fixed schema (defined before inserting data)	Flexible schema (can change, schema-less in many cases)
Query language	SQL (Structured Query Language)	Varies by database (e.g. query APIs, JSON-based queries)
Data structure	Structured and highly organised	Semi-structured or unstructured
Scalability	Typically scales vertically (bigger machines)	Designed to scale horizontally (across many machines)
Transactions	Strong ACID compliance (Atomicity, Consistency, Isolation, Durability)	Often follows BASE (Basically Available, Soft state, Eventually consistent)
Best for	Complex queries, structured data, transactional systems	Large volumes of data, high scalability, rapidly changing data

Example: PostgreSQL (Relational Database)

PostgreSQL

id	name	email	created_at
1	John	john@example.com	2025-09-10
2	Mary	mary@example.com	2025-09-11
3	Tunde	tunde@example.com	2025-09-12

Data is stored in structured tables with a fixed schema.

Example: MongoDB (NoSQL Database)

MongoDB

```
{
  "_id": "68c1a9e2",
  "name": "John",
  "email": "john@example.com",
  "age": 28,
  "skills": ["Python", "SQL", "AWS"]
}
```

Data is stored as flexible documents. Each document can have different fields.

Key Takeaway

Use a relational database when you need structured data and complex queries.
Use a NoSQL database when you need flexibility, scalability and can work with semi-structured or unstructured data.

Tech365
Leveraging technology for growth

LEARN | BUILD | GROW

Managed Databases

Managed databases are fully managed database services provided by cloud providers, where the infrastructure, maintenance and many administrative tasks are handled for you.

The Provider Manages

- Provisioning and setup
- Patch management and software updates
- Backups and recovery
- High availability and failover
- Scaling (in many cases)
- Infrastructure and hardware
- Monitoring and health checks

Key Benefits

- Less administration
- High availability and reliability
- Easy scaling
- Built-in security features
- Focus on your applications and data

When to Use Managed Databases

- You want to reduce operational overhead
- You need high availability and reliability
- You want to scale quickly
- You prefer to focus on building your applications
- You need built-in backup, monitoring and security

Examples of AWS Managed Databases

AWS provides managed database services for different use cases.

Amazon RDS
Managed relational database (MySQL, PostgreSQL, etc.)
Relational (OLTP)

Amazon Aurora
High-performance relational database (MySQL, PostgreSQL)
Relational (OLTP)

Amazon DynamoDB
Fully managed NoSQL key-value and document database
NoSQL

Amazon Redshift
Fully managed data warehouse for analytics
Analytical (OLAP)

Amazon Neptune
Managed graph database
Graph

Amazon DocumentDB
Managed document database (MongoDB compatible)
Document (NoSQL)

Amazon Timestream
Managed time-series database
Time-series

Amazon ElastiCache
Managed in-memory database (Redis, Memcached)
In-memory

Key Takeaway

Managed databases help you run reliable, secure and scalable databases without worrying about the underlying infrastructure. You focus on your data and applications, while AWS handles the rest.

Tech365
Leveraging technology for growth

LEARN | BUILD | GROW

AWS

What is Amazon RDS?

Amazon RDS (Relational Database Service) is a fully managed database service from AWS that makes it easy to set up, operate and scale a relational database in the cloud.

Supported Database Engines

Amazon RDS supports popular relational databases, including:

PostgreSQL | MySQL | MariaDB | Oracle | Microsoft SQL Server

Key Benefits

- Fully managed (no manual installation, patching or hardware management)
- Automatic backups and point-in-time recovery
- High availability and fault tolerance
- Easy scaling (storage and compute)
- Built-in security features
- Cost-effective and reliable
- Supports multiple database engines

Common Use Cases

- Web and mobile applications
- Business applications (CRM, ERP, etc.)
- Databases for small and medium businesses
- Development and testing environments

Key Takeaway

Amazon RDS lets you focus on your applications and data, while AWS handles the infrastructure, maintenance and many of the operational tasks.

Tech365
Leveraging technology for growth

LEARN | BUILD | GROW

PostgreSQL and MySQL

Two popular open-source relational databases with different strengths.

Both are relational databases
They store data in tables, use SQL and are widely used in web applications, business systems and data engineering pipelines.

	PostgreSQL Powerful. Flexible. Open source.	MySQL Fast. Reliable. Widely used.
Developer	PostgreSQL Global Development Group	Oracle (originally developed by MySQL AB)
Type	Open-source relational database	Open-source relational database
First released	1996	1995
Focus	Standards compliance, advanced features and extensibility	Performance, simplicity and ease of use
SQL standard compliance	High (very close to SQL standards)	Moderate
Extensibility	Highly extensible (custom functions, data types, extensions)	Limited extensibility
Performance	Excellent, especially for complex queries and large datasets	Very good, optimised for high-speed read operations
Usability	Steeper learning curve	Easier to learn and use
Common use cases	Complex applications, analytics, GIS, data engineering	Web applications, content management systems, high-throughput systems
Popular in	Enterprises, data engineering, advanced applications	Web development, startups, small and medium businesses

Common Use Cases for PostgreSQL

- Data engineering and analytics
- Geospatial data (PostGIS)
- Complex applications with advanced data models
- Projects that require high SQL compliance
- Custom extensions and specialised functionality

Common Use Cases for MySQL

- Web applications (e.g. WordPress, Laravel)
- Content management systems
- E-commerce platforms
- High-volume read operations
- Small and medium business applications

Key Takeaway

Both PostgreSQL and MySQL are excellent databases. Choose PostgreSQL when you need advanced features and flexibility. Choose MySQL when you want simplicity and high performance for web applications.

Tech365
Leveraging technology for growth

LEARN | BUILD | GROW

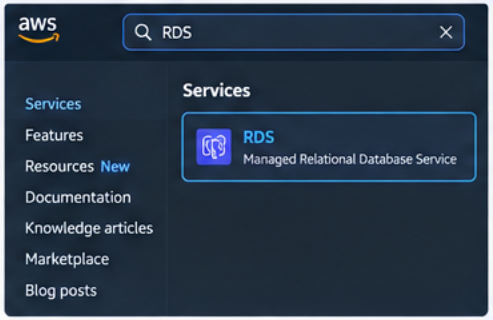
Creating an RDS Database

Step by step guide to create an Amazon RDS database using the AWS Management Console.

Amazon RDS
Managed databases in the cloud

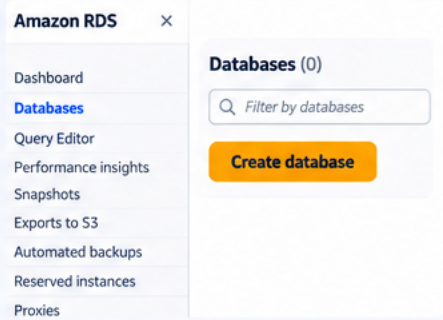
1 Open the RDS Console

Sign in to the AWS Management Console and go to **RDS**.



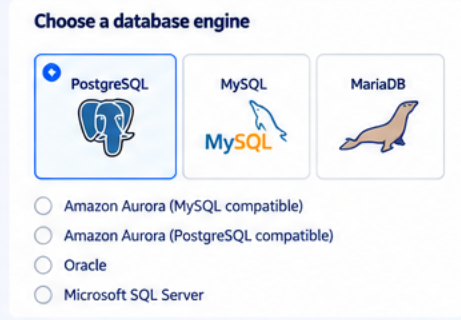
2 Choose Create database

Click on **Create database** to start the setup process.



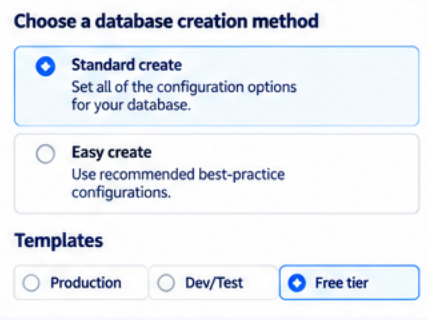
3 Choose a database engine

Select your database engine (e.g. PostgreSQL or MySQL).



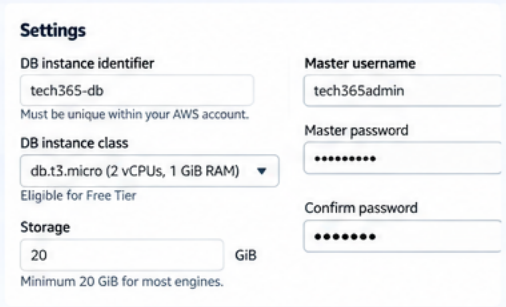
4 Select a template

Choose a template. For learning or testing, you can use the **Free tier** template.



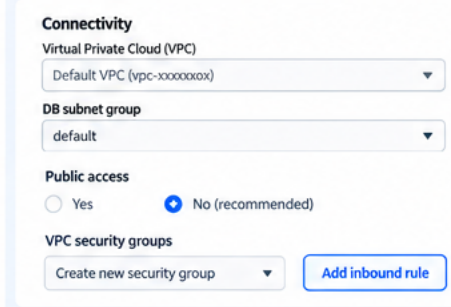
5 Configure settings

Specify important settings such as DB instance identifier, instance class, storage and credentials.



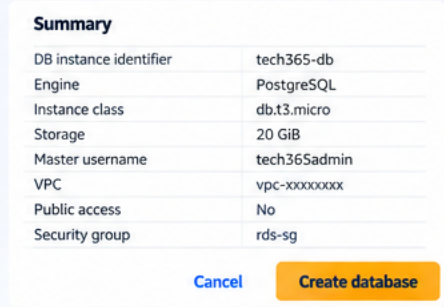
6 Configure connectivity

Choose a VPC, subnet group and configure security group access.



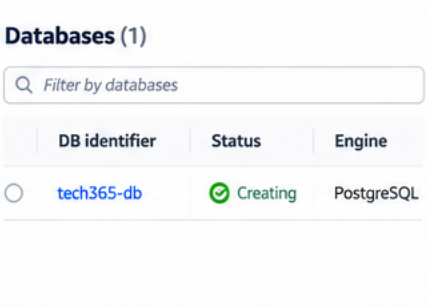
7 Review and create

Review your configuration and click **Create database**.



8 Database is being created

AWS will provision the database. This may take a few minutes.



Key Takeaways

- Amazon RDS makes it easy to create and manage a database in the cloud.
- Choose the right engine (PostgreSQL, MySQL, etc.) for your use case.
- Configure security groups properly.
- You can start with the Free tier for learning and testing.
- AWS handles provisioning, patching, backups and maintenance.
- You only pay for what you use.

Tech365
Leveraging technology for growth

LEARN | BUILD | GROW

Amazon RDS Multi-AZ

Higher availability and reliability for your database.

Amazon RDS
Managed databases in the cloud

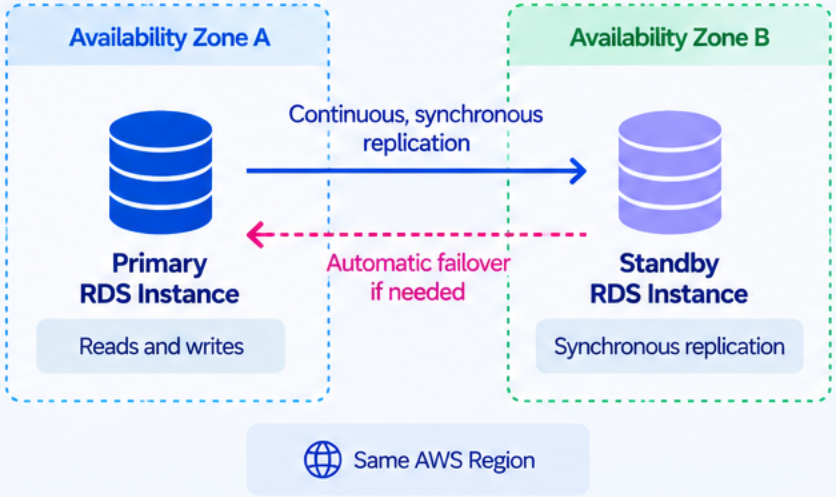
Multi-AZ automatically creates and maintains a standby database in a different Availability Zone to provide high availability and failover.

What is Multi-AZ?

Multi-AZ (Multiple Availability Zones) creates a synchronous standby replica of your primary database in a different Availability Zone within the same region. If there is a failure, Amazon RDS automatically performs a failover to the standby with minimal downtime.

- Improves availability
- Automatic failover
- Synchronous replication
- Minimal downtime

How Multi-AZ Works



Key Benefits

High availability
Database remains available even if an Availability Zone fails.

Automatic failover
RDS automatically switches to the standby instance.

Minimal downtime
Typically 1–2 minutes for failover in most cases.

No manual management
RDS handles replication, monitoring and failover for you.

When to Use Multi-AZ

- Production databases
- Business-critical applications
- Applications that require high availability
- When you want automated failover and recovery

Considerations

- Slightly higher cost than a single-AZ deployment
- Some read replicas are a better option for scaling reads
- Failover is automatic, but applications should be designed to handle reconnections
- Typically used within the same AWS region

Supported Database Engines

PostgreSQL · MySQL · Oracle · SQL Server · MariaDB

Tech365
Leveraging technology for growth

LEARN | BUILD | GROW

Amazon RDS Read Replicas

Scale read performance and improve availability for your database.

Amazon RDS
Managed databases in the cloud

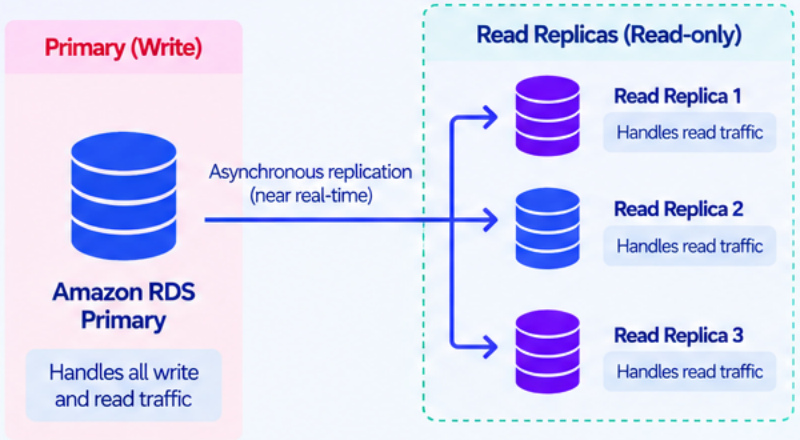
Read replicas are read-only copies of your RDS database that replicate data from the primary database, helping you scale read workloads and improve resilience.

What are Read Replicas?

A Read Replica is a read-only copy of your primary RDS database. It receives data from the primary database using asynchronous replication. You can use read replicas to scale read traffic, improve performance and increase availability.

- Read-only copies of your database
- Asynchronous replication
- Helps you scale read workloads
- Can be in the same or a different AWS Region

How Read Replicas Work



Key Benefits

Better performance
Distribute read traffic across replicas to reduce load on the primary database.

Improved scalability
Add read replicas as your application grows.

Increased availability
Provides redundancy and can be used for disaster recovery.

Flexible deployment
Create read replicas in the same Region or in a different Region.

Common Use Cases

Scale read-heavy applications
Handle more users and queries.

Reporting and analytics
Run reports and analytics without impacting the primary database.

Cross-Region disaster recovery
Create replicas in another Region for additional protection.

Important Notes

- Read replicas are read-only. You cannot write data to them.
- Replication is asynchronous, so there can be a small delay.
- You can have multiple read replicas.
- You can promote a read replica to a standalone database if needed.

Best Practices

- Use read replicas for read-heavy workloads (reports, dashboards, analytics).
- Monitor replication lag to ensure data is up to date.
- Place read replicas in different Availability Zones or Regions for resilience.
- Use appropriate instance classes based on your workload.
- Test failover and promotion procedures.

What is Amazon Aurora?

A high-performance, fully managed, cloud-built relational database engine from AWS.

Amazon Aurora
High performance.
High availability.
Fully managed.

Compatible with MySQL and PostgreSQL.
Built for the cloud.

What is Amazon Aurora?

Amazon Aurora is a fully managed, cloud-built relational database engine within Amazon RDS. It is compatible with MySQL and PostgreSQL and is designed to deliver higher performance, greater scalability and high availability at a lower cost than traditional databases.

Aurora MySQL vs Aurora PostgreSQL

Amazon Aurora is compatible with both MySQL and PostgreSQL, giving you the performance and availability of Aurora with the familiar tools and ecosystems.

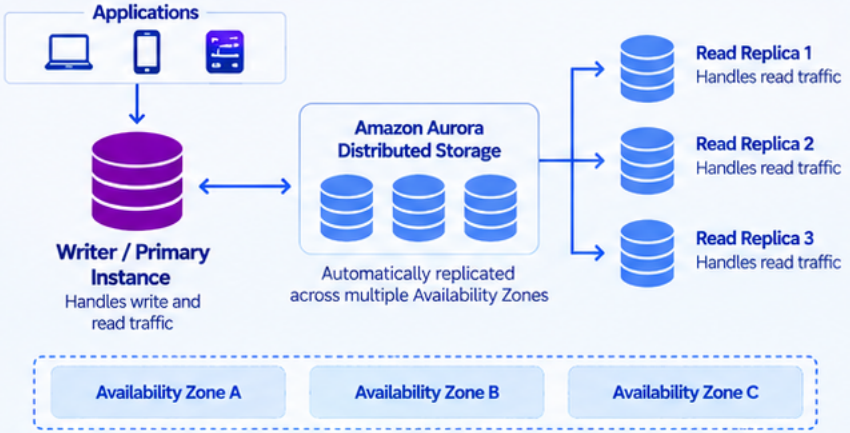
Amazon Aurora MySQL

- Compatible with MySQL
- Supports existing MySQL tools
- Ideal for MySQL workloads

Amazon Aurora PostgreSQL

- Compatible with PostgreSQL
- Supports existing PostgreSQL tools
- Ideal for PostgreSQL workloads

How Amazon Aurora Works



Data is automatically stored in multiple Availability Zones for high durability and availability.

Key Features

- **High performance**
  Up to 5× the performance of standard MySQL and up to 3× that of standard PostgreSQL.
- **Fully managed**
  AWS handles provisioning, patching, backups, monitoring and failover.
- **Automatic storage scaling**
  Storage grows automatically from 10 GB to 128 TB based on your needs.
- **Backups and point-in-time recovery**
  Continuous backups to Amazon S3 with point-in-time recovery.
- **High availability and durability**
  Data is replicated across multiple Availability Zones.
- **Up to 15 low-latency read replicas**
  Scale read workloads with minimal effort.

Common Use Cases

Enterprise applications
Mission-critical business applications

SaaS platforms
Scalable and reliable databases for growing users

High-traffic web and mobile apps
Handle large read and write workloads

Workloads requiring high availability
Applications that cannot afford downtime

RDS vs Aurora

Feature	Amazon RDS	Amazon Aurora
Database engines	Multiple engines (MySQL, PostgreSQL, SQL Server, Oracle, etc.)	MySQL and PostgreSQL compatible
Built for the cloud	Yes	Yes (cloud-native)
Performance	Standard performance	Up to 5× higher performance
Storage scaling	Manual or auto scaling (varies by engine)	Automatic, up to 128 TB
High availability	Multi-AZ (optional)	Built-in, across 3 AZs
Cost	Standard pricing	Better price-performance

Key Takeaway

Amazon Aurora combines the familiarity of relational SQL databases with the performance, scalability and availability of a cloud-native database, making it an excellent choice for modern applications.

Tech365
Leveraging technology for growth

LEARN | BUILD | GROW

Amazon DynamoDB

A fully managed, serverless NoSQL database from AWS.

Amazon DynamoDB
Fast. Scalable. Serverless.

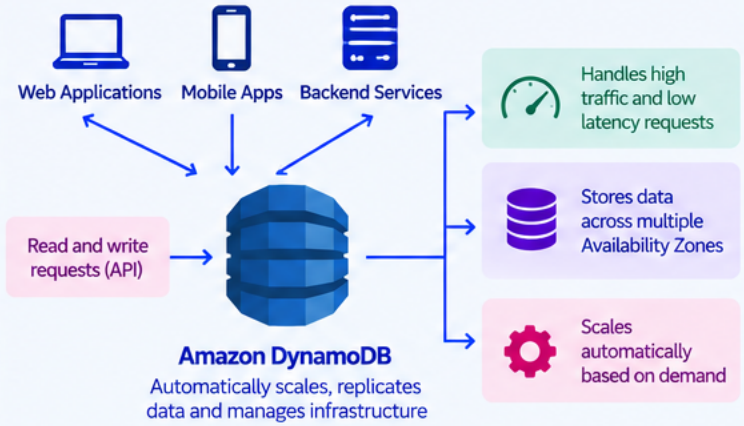
A fully managed NoSQL database designed for high performance and large-scale applications.

What is Amazon DynamoDB?

Amazon DynamoDB is a fully managed, serverless NoSQL database from AWS. It provides fast and predictable performance at any scale, with built-in security, high availability and automatic scaling. DynamoDB is designed for modern applications that need low latency and can handle large amounts of data.

- Fully managed by AWS
- Serverless (no infrastructure to manage)
- High performance and low latency
- Automatically scales to handle any workload
- Highly available and durable

How DynamoDB Works



Key Features

Serverless
No servers to provision or manage. Pay only for what you use.

High performance
Single-digit millisecond latency at any scale.

Automatic scaling
Scales seamlessly to handle millions of requests per second.

High availability and durability
Data is replicated across multiple Availability Zones.

Built-in security
Encryption, IAM integration and fine-grained access control.

Common Use Cases

E-commerce
Product catalogue, shopping cart, user sessions

User management
User profiles, preferences and authentication

Gaming
Player data, leaderboards and game state

IoT and real-time apps
Device data, sensor data and real-time analytics

DynamoDB vs Relational Databases

Feature	DynamoDB	Relational Databases (e.g. RDS)
Database model	NoSQL (key-value, document)	Relational (tables)
Server management	Fully managed, serverless	Managed instances
Scaling	Automatic, seamless	Manual scaling (often vertical)
Performance	Single-digit ms latency	Generally higher latency
Schema	Flexible (schema-less)	Fixed schema
Best for	High-scale, low-latency applications	Complex queries and transactions

Key Takeaway

Amazon DynamoDB is the go-to database for modern, high-scale applications that need speed, scalability and reliability without the operational overhead of managing servers.

Tech365
Leveraging technology for growth

LEARN | BUILD | GROW

Amazon Redshift

Fast, scalable and fully managed data warehouse from AWS.

Amazon Redshift
Fast. Scalable. Fully managed.

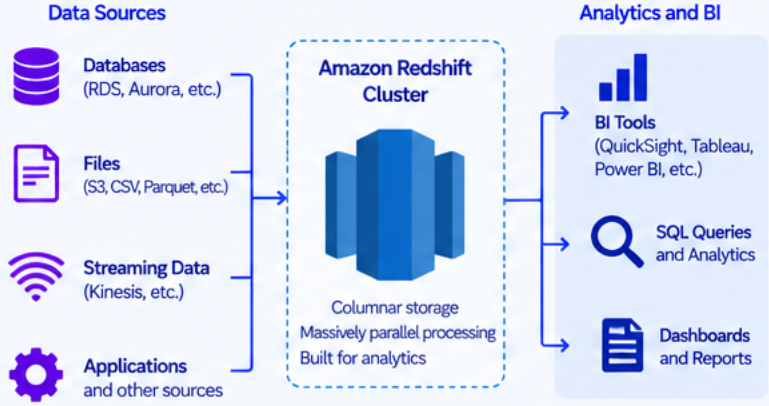
A cloud data warehouse built for analytics, designed to handle large volumes of data and complex queries.

What is Amazon Redshift?

Amazon Redshift is a fully managed, petabyte-scale data warehouse from AWS. It is designed for fast analytics on large datasets using columnar storage, parallel processing and advanced query optimization. It helps organisations gain insights from their data quickly and at scale.

- High performance for analytics
- Scales to petabytes of data
- Fully managed by AWS
- Integrates with the AWS ecosystem
- Cost effective for large scale analytics

How Amazon Redshift Works



Load, store, and analyse large volumes of data with high performance.

Key Features

High performance
Optimized for fast queries on large datasets.

Scalability
Easily scales from gigabytes to petabytes.

Fully managed
AWS handles provisioning, patching, backups and maintenance.

Columnar storage
Stores data by columns for faster query performance.

Integrates with AWS
Works seamlessly with S3, Glue, Athena, QuickSight and more.

Common Use Cases

Business intelligence
Analyse business data and create dashboards.

Data lakes and modern analytics
Combine with S3 and Glue for large scale analytics.

Reporting
Generate fast reports on large datasets.

Machine learning and advanced analytics
Use SQL or integrate with SageMaker for AI/ML workflows.

Redshift vs Other Analytics Options

Feature	Amazon Redshift	Amazon Athena	Amazon RDS
Type	Data warehouse	Query service	Transactional database
Best for	Large scale analytics	Ad-hoc queries on S3 data	OLTP applications
Performance	High (optimized)	Good (serverless)	Good for transactions
Storage	Managed storage	Uses data in S3	Managed storage
Typical data size	GB to PB	GB to PB	MB to TB

Key Takeaway

Amazon Redshift helps you turn large amounts of data into valuable insights faster. It is a powerful, fully managed data warehouse that scales with your business needs.

Tech365
Leveraging technology for growth

LEARN | BUILD | GROW

Amazon Athena

Query your data directly in Amazon S3 using standard SQL.

Amazon Athena
Serverless Query Service

Run SQL queries on data in Amazon S3 without managing servers.

What is Amazon Athena?

Amazon Athena is a serverless, interactive query service that makes it easy to analyse data directly in Amazon S3 using standard SQL. You do not need to set up, manage or scale any infrastructure. You simply point to your data, write SQL queries and get results in seconds.

- Serverless (no infrastructure to manage)
- Uses standard SQL
- Works directly with data in Amazon S3
- Pay only for the queries you run
- Ideal for ad-hoc analysis and exploration

How Amazon Athena Works

Your data
(CSV, JSON, Parquet, ORC, Avro, etc.)

Serverless query service
Processes your SQL queries directly on data in S3

Results returned in seconds
(Use AWS Console, CLI, SDK or BI tools)

No servers. No clusters. Just **SQL.**

Key Features

Serverless
No infrastructure to set up, manage or scale.

Standard SQL
Use familiar SQL to query your data.

Directly query data in S3
Supports CSV, JSON, Parquet, ORC, Avro and more.

Pay per query
You only pay for the amount of data scanned.

Integrates with analytics tools
Works with QuickSight, Tableau, Excel, and more.

Common Use Cases

Data exploration
Quickly explore large datasets in S3.

Business analytics
Run ad-hoc queries for reports and dashboards.

Data lake analysis
Analyse data in your data lake without moving it into a database.

Log analysis
Query application, server or security logs stored in S3.

Example Query

```
SELECT
  year,
  product,
  SUM(sales) AS total_sales
FROM s3_data.sales
WHERE year = 2024
GROUP BY year, product
ORDER BY total_sales DESC;
```

This query reads data directly from files in Amazon S3 and returns the total sales by product for 2024.

Key Takeaway

Amazon Athena makes it simple and cost-effective to analyse data in Amazon S3 using standard SQL, without managing any infrastructure. It is a powerful tool for data analysts, engineers and business teams.

Tech365
Leveraging technology for growth

LEARN | BUILD | GROW

Database Security on AWS

Protect your data. Ensure compliance. Build with confidence.

Secure AWS Databases
People. Access. Network. Data. Monitoring.

Use AWS security features and best practices to keep your database environments safe.

Why Database Security Matters

Your databases store your most valuable data. Proper security helps you prevent unauthorised access, protect sensitive information, meet compliance requirements and maintain business continuity.

- Protect sensitive data
- Prevent unauthorised access
- Meet regulatory and compliance requirements
- Maintain trust with your customers

Layers of Database Security

Key AWS Security Features

IAM
Manage user access with fine-grained permissions and least privilege.

Encryption
Use AWS KMS to encrypt data at rest and in transit (SSL/TLS).

VPC and Security Groups
Restrict database access to trusted networks and IP addresses.

Auditing and Monitoring
Use Amazon CloudWatch, AWS CloudTrail and database audit logs to track activity.

Backup and Recovery
Enable automated backups, snapshots and multi-AZ for high availability.

Network Security

- Place databases in private subnets
- Use security groups to allow only required traffic
- Use VPC endpoints for private access
- Avoid public access to databases

Encryption

- Encrypt data at rest using AWS KMS
- Use SSL/TLS for data in transit
- Enable encryption for RDS, Aurora, DynamoDB, Redshift and other databases
- Manage and rotate encryption keys properly

Access Control

- Use IAM roles and policies
- Apply the principle of least privilege
- Use multi-factor authentication (MFA)
- Manage database users and roles
- Megularly review and remove unused access

AWS Database Security Across Services

Amazon RDS
Encryption, IAM, security groups, audit logs

Amazon Aurora
Encryption, IAM, security groups, audit logs

Amazon DynamoDB
Encryption (KMS), IAM, VPC, Fine-grained access control

Amazon Redshift
Encryption (KMS), IAM, VPC, audit logging

Amazon Athena
Use IAM and Lake permissions to control access to data in S3

Best Practices

- Follow the principle of least privilege
- Use private subnets and security groups
- Enable encryption for all databases
- Monitor with CloudWatch and CloudTrail
- Keep backups and test recovery
- Regularly review access and permissions
- Follow AWS Well-Architected Framework

Key Takeaway

Database security on AWS is a combination of the right configuration, access control, encryption, monitoring and regular review. A well-secured database environment protects your data and keeps your business running with confidence.